

目录

拓扑排序应用软件可行性分析报告.....	2
一 项目目标与分析范围.....	2
二 技术可行性.....	2
1 技术选型与开发基础.....	2
2 图存储与基础算法.....	3
3 多序列生成与规模控制.....	3
4 实施证据与技术边界.....	4
三 经济与资源可行性.....	4
四 操作与协作可行性.....	5
1 用户操作可行性.....	5
2 人员分工与协作基础.....	6
五 进度可行性.....	6
六 主要风险与应对.....	7
七 综合结论与依据.....	8

拓扑排序应用软件可行性分析报告

本项目采用 Java 与 Swing 开发本地桌面拓扑排序软件，在现有人员分工、设备和课程期限下具备实施可行性。当前集成版本已经具备关系输入、建图、算法计算、可视化及导出能力，核心算法和数据处理的复跑结果支持所选技术路线。项目可行的主要条件是控制多序列枚举规模、保持跨模块接口一致，并以实际交付版本核对运行环境和使用说明，不能将限量结果或单机验证扩大为无限规模及所有平台的保证。

本文由组员 A 骆深敏编写，版本为 V1.0，定稿日期为 2026 年 9 月 23 日。

一 项目目标与分析范围

项目需要将课程之间的先修后修关系转换为有向图，显示关系结构，并给出尽可能多的合法学习顺序。软件还需支持手动输入、文本文件导入、关系图与排序结果导出，以及对错误输入和循环依赖的处理。任务书要求使用图形界面且不能采用网页形式，因此本项目选择桌面应用作为交付形态，以本地文本文件承载数据。

可行性分析关注实现路径、资源条件和交付风险，不重复逐个类的详细设计，也不替代独立测试报告。数量上限 1000、GUI 超时 30 秒等配置属于团队实施方案，并非教师规定的统一性能指标。对于课程数据，还需要区分软件正确处理输入与输入是否准确反映培养方案这两件事，算法结果正确不能自动证明原始关系整理没有遗漏。

二 技术可行性

1 技术选型与开发基础

Java 标准库能够提供集合、文件读写、线程管理及图形界面所需能力，Swing 可以实现窗口、表格、菜单、列表和自绘画布。当前项目使用这些能力完成主要功能，没有引入额外数据库、服务器或第三方绘图库。技术依赖集中在 JDK，有利于成员共享开发环境，也降低现场准备多个组件的复杂程度。

图模型、算法、解析、绘图和界面控制按职责分包，各模块通过公共接口连接。核心算法不依赖 Swing，可以先用程序化测试确认行为，再接入图形界面；输入解析直接返回统一的 Graph，减少不同模块重复建图的偏差。现有分工和独立测试结果表明这种拆分能够支持多人并行开发，主要集成成本来自接口语义和状态衔接，而不是缺少基本技术实现路径。

当前经过实际编译和运行验证的基准为 Windows 与 JDK 21.0.6。main 中的运行说明仍有“兼容 JDK 8”文字，但源码使用模式匹配等较新语言特性，本轮 Java 8 目标编译失败，Java 16 目标交叉编译通过。交叉编译不能代替低版本运行环境测试，因此当前部署按 JDK 21 说明，避免在技术可行性结论中承诺尚未验证的兼容范围。

2 图存储与基础算法

课程与先修关系能够直接映射为有向图，Graph 使用节点映射及前驱、后继集合保存关系，支持稳定遍历和度数查询。该结构既能处理普通依赖，也能表示孤立节点、自环和多个不连通部分，并在重复边输入时保持度数正确。查询返回不可修改的快照，算法使用局部状态，能够降低一次计算影响后续调用或画布数据的风险。

基础排序使用 Kahn 算法，通过处理零入度节点得到一种合法顺序；处理数量不能覆盖全图时判定存在环。环路径定位结合 Kahn 预判和三色深度优先遍历，返回具体的闭合节点路径，便于界面解释循环依赖。图存储量与节点、边数量相关，单序列排序和环检测采用 $O(V+E)$ 的图遍历路线，其中 V 为节点数、 E 为边数，技术复杂度适合本课程项目。

3 多序列生成与规模控制

多序列生成采用显式回溯，对当前可以选择的零入度节点分别展开，并在返回上一层时恢复临时入度和选择状态。实现避免直接依赖递归调用栈，提供数量上限、取消和超时参数，返回已生成结果及其完整性。小图全结果对照和图 1 的独立计数验证，为该路线的正确性提供了依据。

完整枚举的主要限制在于结果本身可能非常多，例如 n 个互不约束的节点存在 $n!$ 种顺序。即使节点和边都能存入内存，也不代表全部结果可以在可接受时间内生成和保存；无递归栈风险也不等于没有内存压力。因而本方案将限制搜索规模、保留已完成序列以及明确停止原因作为必要措施，不承诺任意千节点图都能快速完整枚举。

GUI 默认最多生成 1000 条序列，并设置 30 秒计算时限，满足常见课堂展示中查看多种结果的需要。API 允许用 0 表示不设数量或时间限制，但使用者应根据图形结构和内存条件选择参数；当前界面没有因此自动具备可配置的“全部结果”开关。只有状态为 COMPLETED 且完整标记为 true 时，结果数量才可作为确切总数使用。

4 实施证据与技术边界

本轮在 main 固定版本上完成全部源程序和测试源码的普通编译，A 的四组作者自测共 79 项通过。E 的算法测试 18 项通过，解析主用例 29 项通过、1 项跳过，一致性检查 15 项通过；D 的解析、预检、文件和数据测试分别通过 35、39、14、10 项。不同来源的测试分别保留归属与统计，不能将其中的跳过项写成已经覆盖，也不能将程序化测试视为完整人工验收。

任务书图 1 的输入包含 15 个节点、16 条边，在不限数量、30 秒时限下完整生成 31,185 条序列，与独立子集计数一致。当前课程数据包含 43 个节点、85 条边，解析和无环检查通过，默认限量枚举正常返回。两组输入在隐藏窗口的程序化 GUI 检查中均经过实际计算按钮完成后台处理，并成功按两种布局生成 PNG，支持当前主流程的技术可运行性。

三 经济与资源可行性

系统主要使用成员已有电脑、现有 JDK 与开发工具完成开发和验证，采用本地文件读写，不需要为基本功能另建服务器或部署数据库。当前实现没有引入必须新增采购的专用硬件或第三方运行组件，技术方案不会主动增加这类支出。这里描述的是实施所需资源，并未调查所有成员的工具授权和个人支出，因此不以“无新增组件”推导整个项目实际费用绝对为零。

项目的主要投入是成员学习、编码、联调、测试和文档整理所需的时间。统一公共接口和运行环境能够减少重复适配，保留可复现的输入及测试记录能够降低定位问题的成本，明确各模块责任则有助于避免多人重复修改同一部分。与临近验收时重新更换框架相比，在已运行的技术方案上集中修复缺陷和统一说明更符合现阶段资源条件。

存储资源主要用于源程序、文本数据、排序结果、图形及演示材料。对于全枚举，应先评估结果数量，再决定是否保存全部序列，避免让大量结果占用不必要的内存和磁盘；普通运行则使用限量配置。临时验证副本和个人资料与正式交付分开存放，最终包仅携带运行和验收所需内容，有利于控制文件规模并提高检查效率。

四 操作与协作可行性

1 用户操作可行性

图形界面将输入、关系图、结果列表和状态信息集中展示，用户可以按“输入或导入、计算、查看、导出”的顺序完成任务。文本输入符合任务书关系对格式，表格编辑为录入提供辅助，关系图和排序列表帮助用户理解不同合法顺序。循环依赖通过具体路径和高亮反馈，错误输入通过行号与原因提示，能够比只提供控制台算法更适合课堂展示和普通使用。

软件仍要求使用者理解有向关系的含义，尤其是 `<a,b>` 表示 a 必须先于 b，而不是两者可以任意排序。状态栏和导出文件需要明确区分全部结果、数量限制、取消和超时，避免将“已生成 1000 条”理解成“总共 1000 条”。用户手册应使用当前文件名、入口和界面描述，不能把测试脚本写成 GUI 启动器，也不能沿用已经不适用的 Java 版本要求。

运行环境应具有可用桌面、能够显示中文的字体、可读的输入文件和可写的导出目录。源码与文本按 UTF-8 处理，编译明确指定编码；界面主要使用 Microsoft YaHei，文本区域使用逻辑字体，现场需要检查实际显示效果。本机运行支持该方案可用，但是否在干净验收电脑或其他操作系统同样顺利，仍需通过实际部署检查确认。

2 人员分工与协作基础

当前由 B 负责 GUI、交互控制和项目统筹，A 负责图模型、算法、接口规范及相关报告，C 负责可视化与图片导出，D 负责解析、文件处理及相关文档，E 负责独立测试和交付材料。各成员已有对应代码或文档成果，说明任务拆分具有落实基础。

公共接口契约统一了图遍历方式、解析结果、环路径和枚举状态，分支开发使成员能够分别维护自己的成果。联调时发现的问题可以通过固定输入、版本和预期行为向对应成员反馈，修复后复跑相同场景确认结果。A 与 D 的名称边界核对、与 C 的图片导出复核、与 B 的结果状态对齐以及与 E 的测试口径核对，均体现了这种协作方式的实际作用。

多人分工也意味着单个模块自测通过不能替代整体衔接验证。接口或数据格式发生变化时，需要同步调用方、测试和文档；成员提供的检查结论应保留原版本与操作条件。技术能力与任务分配基本匹配的同时，仍需持续投入沟通和集成检查，才能将独立成果转化为稳定的软件交付。

五 进度可行性

本项目将 9 月 16 日至 27 日作为主要实施阶段，共安排 12 天，并于 9 月 28 日进行最终提交与现场验收。计划依次推进设计、编码、集成、测试和材料整理，整体顺序符合软件开发中先明确需求与接口、再实现和验证的基本过程。考虑到开发周期较短，各阶段之间需要适当交叉，文档和测试应随开发同步开展，不能全部留到最后完成。

前 3 天主要完成技术方案、公共接口、概要设计和详细设计，为成员并行开发提供共同依据。第 4 至第 5 天集中完成核心模块初版，并尽早接通输入、建图、计算和显示的基本流程，为阶段性检查提供可演示的成果。第 6 至第 8 天继续完善交互、可视化、导出和真实课程数据应用，逐步稳定功能范围；第 9 至第 11 天重点进行系统测试、缺陷修复和回归验证，第 12 天集中核对文档、演示材料和最终交付包。

这一安排具有可行性的主要原因是任务能够按照模块分工并行推进，而不必由所有成员依次完成。图模型和公共接口确定后，算法、界面、绘图、解析及测试准备可以同步开展，减少相互等待。通过先形成基本可运行版本，再逐步完善功能，可以较早发现模块衔接问题，使后续测试阶段有时间修正，而不是在提交前才首次集成。

不过，12 天的计划较为紧凑，能否按期完成取决于接口确认、阶段交付和问题反馈是否及时。应优先保证任务书要求的基本功能，后期控制新增功能，并利用测试阶段处理缺陷和必要调整；若前期发生延误，应先压缩非必要扩展，保留基本测试与交付核对时间。总体而言，在成员按分工并行开发、持续沟通并同步维护文档的条件下，该计划具有实施可行性，但时间余量有限，需要通过阶段检查及时发现和处理进度偏差。

六 主要风险与应对

首先是枚举规模风险，课程关系越少，可能的合法顺序通常越多，完整搜索与结果存储可能迅速增加。当前通过数量限制、时限、取消和完整性状态控制这一风险，测试需同时记录节点数、边数、图结构及限制参数。千节点链、千个孤立节点和随机 DAG 的表现代表不同条件，不能用某一次耗时作为所有千节点输入的性能承诺。

其次是模块状态与异步衔接风险，取消之后的后台回调、连续计算、输入修改及导出都可能影响当前结果的含义。算法返回的完整性必须与对应序列一起传递，界面需要确保旧任务不会覆盖新任务，也不能让界面显示与导出元数据错配。当前源码和已有反馈为进一步核对提供了基础，但本轮隐藏窗口正常流程检查未复测所有取消和超时时序，仍应以专项回归补足。

第三是数据来源与应用解释风险，软件即使正确执行，也可能忠实计算一份整理不准确的课程关系。图 1 方向按团队既有的先修到后修解释采用，课程数据的文件注释说明来源与映射，但本轮没有逐条重新比对培养方案原文。交付时应保留原始依据和整理说明，课程代码、替代代码及被排除课程的含义由数据提供者解释，避免将软件展示扩大为官方选课指导。

第四是环境与交付说明风险，当前 main 的 JDK 8 兼容描述已经被编译核查否定，且未跟踪成品 JAR 或专用 GUI 启动 bat。应统一使用已验证的 JDK 21 与源码启动方法；如果团队另行提供 JAR 或 bat，需要将真实文件、目录和命令同步给用户手册编写者。`run_tests.bat` 只能说明存在测试入口，不能据此宣称软件已经具备完整的一键分发包。

最后是材料与进度风险，源码更新后如果仍引用旧截图、旧接口或旧测试条件，会降低报告的可信度和复现性。各份文档应写清其代码基准，区分历史验证与本轮复跑，成员实现、作者自测、独立测试和联调工作分别归属。最后阶段集中处理影响交付的差异，并按实际文件检查材料齐全，比继续增加未经验证的功能更有利于按期完成。

七 综合结论与依据

综合技术路线、现有资源、人员成果和集成进度，本项目能够以 Java Swing 桌面软件的形式完成规定的核心任务，继续采用现有方案具有可行性。已经验证的图模型、算法与数据处理，以及正常 GUI 计算链路，为这一判断提供了具体支持。可行性结论的适用范围是已说明的功能、参数和环境，最终交付仍应完成当前版本的人工回归和运行材料一致性检查。